

Combinatorial Decision Making and Optimization Project Work

Andreea Dornescu andreea.dornescu@studio.unibo.it
Davide Ligabue davide.ligabue@studio.unibo.it
Alessandro Picardi alessandro.picardi@studio.unibo.it

December 2025

Summary

1	Introduction	1
1.1	Circle Method	1
1.2	Objective function	2
1.3	Defective periods	3
1.4	Solution Space Symmetries	3
2	CP Model	4
2.1	Decision variables	4
2.1.1	Auxiliary Variables	4
2.2	Objective function	4
2.3	Constraints	4
2.3.1	Implied Constraints	5
2.3.2	Symmetry Breaking	5
2.4	Validation	5
3	SAT Model	7
3.1	Decision variables	7
3.1.1	Auxiliary variables	7
3.2	Objective function	8
3.3	Constraints	8
3.3.1	Implied constraints	8
3.3.2	Symmetry breaking constraints	9
3.4	Validation	9
4	MIP Model	10
4.1	Decision variables	10
4.1.1	Auxiliary variables	10
4.2	Objective function	10
4.3	Constraints	11

4.3.1	Implied constraints	11
4.3.2	Symmetry breaking constraints	11
4.4	Validation	12
5	Conclusion	12

1 Introduction

Let n be the number of teams (even), we define the following parameters that we'll be used in all the report below:

- number of weeks $W = n - 1$,
- number of periods $P = \frac{n}{2}$,
- number of matches $M = \frac{n \cdot (n-1)}{2}$.

Given that the set of all unordered matches can be represented as a list of $\binom{n}{2}$ pairs $\{(i, j) \mid 1 \leq i < j \leq n\}$ ordered lexicographically, we define a bijective monotone function:

$$\begin{aligned} match_ID : \{(i, j) \mid 1 \leq i < j \leq n\} &\longrightarrow \{1 \cdots M\}, \\ match_ID(i, j) &= ((i - 1) \cdot n + j) - \frac{i \cdot (i + 1)}{2}, \end{aligned} \quad (1)$$

that assigns to each unordered team pair a unique match ID.

From now on, when we use a quantification over some index $m \in \{1 \cdots M\}$ that refers to a match, we are actually referring to the unique match ID that identified such match.

We will use two lists $T1$ and $T2$ of size M such that:

$$T1[m] = i, T2[m] = j, \quad \forall m = match_ID(i, j) \in \{1 \cdots M\}, \quad (2)$$

this formulation avoids repeated function calls and instead relies on direct array lookup, a construct that is highly optimized in constraint solvers.

1.1 Circle Method

The initial schedule is given by the circle method (see [1]) which very efficiently allows us to compute a schedule that contains all the matches to be played and insert them so that they played exactly one game a week. The following is a brief description on how the circle method works for generating a feasible schedule w.r.t. the constraints defined below (in [1] there is a full analysis of this method, including also the proofs of several theorems about the correctness of the solution generated by the circle method). The circle method initially selects a team, let us say team n , and for each week $w \in W$ this team will face the team labeled w . The remaining matches in each week w are established by the following modular equation:

$$“i \text{ vs } j” \text{ in week } w \Leftrightarrow i + j \equiv 2w \pmod{n - 1}. \quad (3)$$

This is sound since the function $w \in \mathbb{Z}_{N-1} \mapsto 2w \in \mathbb{Z}_{N-1}$ is bijective iff N is even.

We can visualize this by imagining a circle where all teams (from 1 to $n - 1$) are placed following their numerical order, while the team n is fixed at the

center. For each week, the pairs are selected based on their position. Team n plays against a specific team from the circle (as stated before), from that point we move in opposite directions along the circle to form the remaining pairs. For instance, the next pair would be the two teams immediately to the left and right from the starting point (the team that faces the team n in the current week), then the two teams positioned further out, and so on.

This only leaves open the problem of arrange all matches within the schedule of period in a fashion that satisfies the period participation constraint. The resulting space of possible arrangements has cardinality $(n/2)!^{n-1}$, given by the direct product of the Symmetric permutation Groups of each period.

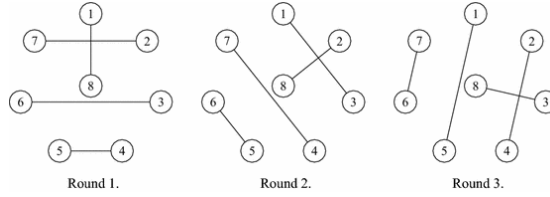


Figure 1: Circle Method Visualized

1.2 Objective function

For the optimization sub-problem the idea implemented in every method is to define for each team a function that quantifies its deviation from an ideal home/away balance, that is when the count of home games is one more or one less than the count of away games, resulting in the following partial objective function:

$$\begin{aligned} \forall t \in \{1 \dots n\} : \quad & imbalance(t) \in \{1, \dots, n-1\}, \\ \forall t \in \{1 \dots n\} : \quad & imbalance(t) = |home_count(t) - away_count(t)| = \quad (4) \\ & = |2 \cdot home_count(t) - (n-1)|, \end{aligned}$$

where $\forall t \in \{1 \dots n\}$, $home_count(t) \in \{1 \dots W\}$, $away_count(t) \in \{1 \dots W\}$ are the total number of home and away games played by team t . By definition, $\forall t \in \{1 \dots n\}$, $home_count(t) + away_count(t) = n-1$. Computationally:

$$home_count_t = \sum_{m=1}^N ((T1[m] == t) (1 - swap[m]) + (T2[m] == t) swap[m]). \quad (5)$$

The objective function to minimize is: $\|imbalance\|_{\ell_\infty} = \max(imbalance)$. This makes it so that imbalance is set to decrease uniformly with respect to each team.

1.3 Defective periods

Some constraints are based on the concept of a “defective period” for a given team. To formalise this, we need the following lemma.

Lemma 1. *Given a feasible schedule, there exists a surjective function $\delta : \{1 \cdots n\} \mapsto \{1 \cdots P\}$ assigning each team to a unique **defective period**, namely a period in which the team appears only once. Each period’s preimage contains exactly two teams (each period has exactly two teams that map to it via δ).*

Fix a period $p \in \{1 \cdots P\}$. Each period contains $W = n - 1$ matches, and therefore $2(n - 1)$ total team appearances.

Each team can appear in any period at most twice, so the maximum number of appearances across all n teams in a period is $2n$. Since only $2(n - 1)$ slots are used, exactly two team appearances are missing.

Suppose both missing appearances belonged to the same team t . Then t plays 0 times in period p , and must play all $n - 1$ matches in the remaining $P - 1$ periods. But since each team appears at most twice per period, this gives at most $2(P - 1) = 2(n/2 - 1) = n - 2$ appearances, which is insufficient.

Therefore, the missing appearances must belong to *distinct* teams, each appearing exactly once in p . Since there are $n/2$ periods and each team is defective in exactly one of them, the mapping δ is a surjective function, and each codomain pre-image contains exactly two teams. ■

1.4 Solution Space Symmetries

In the generic formulation of the round-robin scheduling problem, every feasible solution remains invariant under the action of the group $S_{n/2} \times S_{n-1} \times S_n$, corresponding respectively to permutations of periods (rows), weeks (columns), and team labels. These three permutation groups act independently and commute, yielding a direct product structure. The resulting number of isomorphic schedule representations is therefore $n! \cdot (n-1)! \cdot (\frac{n}{2})!$. We will fashion our search space to have just one representative for each solution class.

Restricting attention to circle schedules, we prune away much of our search space, but electing a canonical representative becomes easier. The last step is enforcing an additional constraint for symmetry breaking:

- 1) fixing team n as the central team,
- 2) requiring it meets teams $1, 2, \dots, n - 1$ in that order across the weeks, and
- 3) canonically fixing the period of matches within the first week.

From this construction, any nontrivial period permutation, team relabelling, or outer-cycle automorphism would violate one of these constraints. Hence, the stabilizer of the resulting schedule is trivial, and every circle schedule collapses onto the same canonical representative.

This canonization drastically reduces redundancy, ensuring that the solver explores only one representative per equivalence class of isomorphic schedules. The resulting space of possible schedules has cardinality $(n/2)!^{n-2}$, a dramatic reduction from the initial $(n^2 - n)!$.

2 CP Model

2.1 Decision variables

We define the following decision variables:

- $\forall w \in \{1 \dots W\}, p \in \{1 \dots P\} : \text{match_vars}_{w,p} \in \{1 \dots M\}$.
The unique identifier for the match scheduled in week w , period p .
- $\forall m \in \{1 \dots M\} : \text{swap}_m \in \{0, 1\}$.
A boolean variable indicating whether the home / away designation of match m is flipped from its lexicographic order. Exclusive to COP phase.

The solution of the problem and the optimisation of its objective functions are two completely independent tasks, only connected by their shared parameter n . This is because the home or away state of a match does not interact with when it will occur in any constraint. If matches were modelled as *ordered* pairs (i.e., including home/away status), the solution space would increase by a factor of $2^M = 2^{n(n-1)/2}$. By decoupling optimization (COP) from the solution structure (CP), we avoid this exponential blow-up, letting the two sub-problem complexities combine additively and not multiplicatively.

2.1.1 Auxiliary Variables

$$\forall w \in \{1 \dots W\}, \forall p \in \{1 \dots P\} : \\ x_{w,p}, y_{w,p} \in \{1 \dots n\}, \text{ where } x_{w,p} < y_{w,p}. \quad (6)$$

The two teams playing in week w , period p . The ordering ensures a unique representation of each unordered match.

2.2 Objective function

The objective function minimizes the Maximum Imbalance (worst-case home-away imbalance across all teams): $\min(\max(\text{imbalance}))$.

2.3 Constraints

Channeling Constraint The channeling constraint connects the team variables $(x_{w,p}, y_{w,p})$ with the match identifier variable $\text{match_vars}_{w,p}$:

$$\forall w \in \{1 \dots W\}, \forall p \in \{1 \dots P\} : \quad x_{w,p} = T1[\text{match_vars}_{w,p}] \wedge \quad (7) \\ \wedge y_{w,p} = T2[\text{match_vars}_{w,p}].$$

Match Uniqueness We require:

$$\text{alldifferent}(\text{match_vars}), \quad (8)$$

this ensures that each match appears exactly once in the schedule.

Circle Schedule To ensure the matches are distributed via the circle method (1.1), the table `week_of_match` : $\{1 \cdots M\} \mapsto \{1 \cdots W\}$ is constructed, mapping each match to its week via a list comprehension.

Then, for each schedule slot, it's imposed that the match filling it is in accordance to the table. Formally:

$$\forall w \in \{1 \cdots W\}, \forall p \in \{1 \cdots P\} : \text{week_of_match}[match_vars_{w,p}] = w. \quad (9)$$

Period Participation Each team may appear at most twice in the same period across the entire tournament:

$$\forall t \in \{1 \cdots n\}, \forall p \in \{1 \cdots P\} : 1 \leq \sum_{w=1}^W ((x_{w,p} == t) + (y_{w,p} == t)) \leq 2. \quad (10)$$

In different scripts this has been implemented both with looping counting constraints and with the dedicated `global_cardinality_low_up` constraint.

2.3.1 Implied Constraints

Defective Constraint We enforce a constraint derived from the functional property of δ , as defined in 1.3: each team has a unique defective period.

$$\forall t \in \{1 \cdots n\} : \sum_{p=1}^P \left(\left(\sum_{w=1}^W ((x_{w,p} == t) + (y_{w,p} == t)) \right) == 1 \right) = 1. \quad (11)$$

2.3.2 Symmetry Breaking

Fixing first week schedule The constraint canonically fixes the periods of match within the first week, as discussed in (1.4):

$$\forall p \in \{1 \cdots P\}, \quad match_vars_{p,1} = match_ID(p, n + 1 - p). \quad (12)$$

Home schedule symmetry breaking Each valid solution to the optimization problem is invariant under the flipping of all the home/away pairs. To half the search space, we fix the first match to have a home/away order that matches the lexicography of the teams.

$$swap_1 = 0. \quad (13)$$

2.4 Validation

Experimental design

The results shown use Gecode 6.3.0 as a solver on a machine with the following hardware specification: Intel(R) Core(TM) i7-12650H 2.30 GHz 16.0 GB RAM.

For all methods, the search consists of two non-interacting components: the integer search for the IDs of the match to be played and the boolean search for the array of swap variables. This consideration allows us to break down the problem in two parts, each agnostic to the other, that only interact in the output section of the MiniZinc script.

The optimisation phase exhibits heavy-tail behaviour characteristic of the exponential $\mathcal{O}(2^{n^2})$ search space: while the majority ($> 90\%$) of instances, even for $n \leq 26$, terminate within 1s, a pathological subset fails to achieve termination within the full 300s timeout. The stochastic nature of `indomain_random` becomes crucial in mitigating these tail cases through restart mechanisms: basic restart strategies proves sufficient to eliminate pathological behaviour, with minimal performance variation across different restart strategies.

Solver-specific limitations constrain the implementation of restarting strategies affecting individual components of a sequential search. As a result, optimal performance is achieved through a search architectures that isolates the restart-sensitive boolean optimization in a separate MiniZinc environment and combines the two search result within the python file that runs the MiniZinc scripts.

Experimental results

Table 1: **Running times** in seconds (rounded down to integers) and **objective function values** for different strategies of CP models. For every value of n and strategy the table reports: time to reach feasibility | objective value, time to reach optimality. Bold objective values denote optimal solutions.

n	global w/out IC	count w/out IC	global w/out SB	count w/out SB	global full	count full
4	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT	UNSAT
6	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
8	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
10	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
12	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
14	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
16	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
18	N/A	N/A	101 1 , 103	5 1 , 6	91 1 , 92	4 1 , 4
20	N/A	N/A	N/A	N/A	95 1 , 97	1 1 , 1

Six strategies are compared, involving two alternative implementations of the period-participation constraints (one using global constraints and one using looping counting constraints), as well as variants of the full model where Symmetry Breaking constraints and Implied constraints (those related to defective periods) are removed.

Results show the role of Implied constraints and Symmetry breaks in narrowing the search space, with two different values of n becoming accessible with their addition.

The result also show a slightly better performance of the naïve implementation of week participation constraint over their `global_cardinality_low_up` version.

3 SAT Model

The SAT model has been built using the Z3 library.

3.1 Decision variables

We define the following variables:

- for the first part, to represent the feasible schedule:

$$\begin{aligned} \forall m \in \{1 \dots M\}, p \in \{1 \dots P\}, \\ match_period_{m,p} = True \Leftrightarrow \text{the match } m \text{ is in the period } p; \end{aligned} \quad (14)$$

- for the optimization part, we use a Boolean variable which tells us if the match under consideration is swapped w.r.t. the previously founded feasible solution:

$$\forall m \in \{1 \dots M\}, swap_m = True \Leftrightarrow \text{the match } m \text{ is swapped.} \quad (15)$$

3.1.1 Auxiliary variables

We use the following variable to represent for each team its defective period, namely the period where the team plays only one match:

$$\begin{aligned} \forall t \in \{1 \dots n\}, p \in \{1 \dots P\}, \\ defective_p_{p,t} = True \Leftrightarrow p \text{ is the defective period for the team } t. \end{aligned} \quad (16)$$

For the optimization part, we need a variable for each team to count its imbalance, and we choose to express this natural number with the unary encoding. Although the more verbose unary representation may generate a larger encoding, its benefits in terms of propagation and quality of the information provided to the SAT solver make it a valid choice (see [2]). This can be done because each value is bounded in a fixed interval: $[0; max_team_imbalance - 1]$.

$$\begin{aligned} \forall t \in \{1 \dots n\}, i \in \{0 \dots max_team_imbalance - 1\}, \\ imbalance_bits_{t,i} = True \Leftrightarrow |imbalance(t)| > i, \end{aligned} \quad (17)$$

where *imbalance* is the function defined in (4). In other words, the number of bits equal to *True* in *imbalance.bit_t* is equal to the value of *imbalance(t)*.

To work correctly, this variable must be followed by ordering constraints:

$$\bigwedge_{t=1}^n \bigwedge_{k=1}^{max_team_imbalance-1} \left(imbalance_bits_{t,k} \rightarrow imbalance_bits_{t,k-1} \right). \quad (18)$$

3.2 Objective function

The Z3 solver in use does not have a SAT optimizer, so the optimization problem is implemented manually as an iterative binary search. At each iteration, if a solution with a better objective values (described in 1.2), i.e. a better maximum imbalance, than the bound arithmetic average is found, *upper_bound* is updated, otherwise *lower_bound* is updated. The iterative process ends when: *lower_bound* > *upper_bound*, *objective_function* = 1 (perfect optimality) or the solver runs out of time.

3.3 Constraints

The following constraints are on the periods of the initial schedule produced by the circle method (see (3)) or are needed to link the initial solution to our variable *match_period* defined in formula (14).

- Each match from the initial schedule must be assigned to exactly one period:

$$\bigwedge_{w=1}^W \bigwedge_{m \in \text{matches_in_week}_w} (\text{exactly_one}(\{\text{match_period}_{m,p} \mid p \in \{1 \dots P\}\})), \quad (19)$$

where *matches_in_week_w* is the set containing all the matches played in the week *w* in the initial schedule produced by the circle method;

- each period must contain exactly one match from every weeks:

$$\bigwedge_{w=1}^W \bigwedge_{p=1}^P (\text{exactly_one}(\{\text{match_period}_{m,p} \mid m \in \text{matches_in_week}_w\})), \quad (20)$$

- every team plays at most two games in the same period:

$$\bigwedge_{p=1}^P \bigwedge_{t=1}^n (\text{at_most_two}(\{\text{match_period}_{m,p} \mid m \in \{1 \dots M\} \wedge (T1[m] = t \vee T2[m] = t)\})), \quad (21)$$

where *T1* and *T2* are the lists defined in (2).

3.3.1 Implied constraints

As proved in Lemma (1), each team has exactly one defective period:

$$\bigwedge_{t=1}^n (\text{exactly_one}(\{\text{defective_p}_{p,t} \mid p \in \{1 \dots P\}\})). \quad (22)$$

3.3.2 Symmetry breaking constraints

- We fix all the first week schedule, maintaining the one decided by the circle method:

$$\bigwedge_{p=1}^P (\{match_period_{m,p} = True \mid m \in matches_in_week_1\}); \quad (23)$$

- we fix the first game to maintain the initial choice between home and away given by the feasible solution:

$$swap_1 = False. \quad (24)$$

3.4 Validation

Experimental design

All the experiments for this part were conducted on the same machine with the following hardware specification: AMD Ryzen AI 7 350, 32 GB RAM. For better reproducibility we used the same seed for all the following produced solutions.

Experimental results

In the following table, we report all the results obtained for the SAT model with different strategies: with (or without) implied constraints IC and with (or without) symmetry breaking SB.

Table 2: **Running times** in seconds (rounded down to integers) and **objective function values** for different strategies of SAT models. For every value of n and strategy the table reports: time to reach feasibility | objective value, time to reach optimality. Bold objective values denote optimal solutions.

n	w/out IC + w/out SB	IC + w/out SB	w/out IC + SB	IC + SB
4	UNSAT	UNSAT	UNSAT	UNSAT
6	0 1, 0	0 1, 0	0 1, 0	0 1, 0
8	0 1, 0	0 1, 0	0 1, 0	0 1, 0
10	0 1, 2	0 1, 2	0 1, 2	0 1, 2
12	0 1, 8	0 1, 8	0 1, 8	0 1, 8
14	0 1, 17	1 1, 18	0 1, 17	1 1, 18
16	5 1, 35	6 1, 36	5 1, 35	4 1, 34
18	7 1, 59	36 1, 87	76 1, 125	13 1, 66
20	N/A	267 9, 300	171 1, 258	116 1, 203

The differences between all strategies are progressively more marked for high values of n . In particular, we can state that both IC and SB make a huge difference for our solver to reach the solution.

4 MIP Model

The Mixed Integer Linear Programming model has been built using the library PuLP [3] and tested with three different solvers.

4.1 Decision variables

- We experimented with both an integer formulation (M variables) and a binary formulation (MP variables); despite the increased number of variables, the binary approach proved significantly faster and more stable, leading us to adopt it as our final model, a match period variable:

$$\begin{aligned} \forall m \in \{1 \cdots M\}, p \in \{1 \cdots P\}, x_{m,p} \in \{0, 1\}, \\ x_{m,p} = 1 \Leftrightarrow \text{the match } m \text{ is in the period } p; \end{aligned} \quad (25)$$

- for the optimization part, we use a Boolean variable which tells us if the match under consideration is swapped w.r.t. the previously founded feasible solution:

$$\forall m \in \{1 \cdots M\}, \text{swap}_m = 1 \Leftrightarrow \text{the match } m \text{ is swapped.} \quad (26)$$

4.1.1 Auxiliary variables

To better express the constraint on the periods of the schedule, we use the following variable to represent the defective period for each team:

$$\text{defective}_{t,p} = 1 \Leftrightarrow \text{team } t \text{ appears exactly once in period } p. \quad (27)$$

To facilitate the formulation of the objective function, we introduce an auxiliary variable for each team to represent its home/away game imbalance. This is necessary to linearize the absolute value present in the *imbalance* function (see (4)).

$$\forall t \in \{1 \cdots n\}, \text{imbalance}_t = b_t^+ + b_t^- \geq 0, \quad (28)$$

in which b_t^+ and b_t^- indicate non-negative auxiliary variables that implement $\max(\text{home}_t - \text{away}_t, 0)$ and $\max(\text{away}_t - \text{home}_t, 0)$ respectively, following standard MIP practice for absolute value linearization. The variable imbalance_t thus represents the absolute difference $|\text{home}_t - \text{away}_t|$.

For the purpose of minimising the ℓ_∞ norm of the imbalance vector we introduce a maximum imbalance auxiliary variable $\text{max_imb} \in \{1, \dots, P\}$ which bounds all individual team imbalances:

$$\forall t \in \{1 \cdots n\}, \text{imbalance}_t \leq \text{max_imb}, \quad (29)$$

4.2 Objective function

The objective function minimizes the worst-case home-away imbalance across all teams (as described in 1.2):

$$\min(\text{max_imb}). \quad (30)$$

4.3 Constraints

- Each match from the initial schedule must be assigned to exactly one period:

$$\forall m \in \{1 \dots M\}, \quad \sum_{p=1}^P x_{m,p} = 1; \quad (31)$$

- each period must contain exactly one match from each week:

$$\forall w \in \{1 \dots W\}, \forall p \in \{1 \dots P\}, \quad \sum_{m \in \text{matches_in_week}_w} x_{m,p} = 1; \quad (32)$$

where matches_in_week_w is the set of all matches in week w from the circle schedule;

- every team plays at most two matches in the same period:

$$\forall t \in \{1 \dots n\}, \forall p \in \{1 \dots P\}, \quad \sum_{m: T1[m]=t \text{ or } T2[m]=t} x_{m,p} \leq 2; \quad (33)$$

where $m \in \{1 \dots M\}$ and $T1$ and $T2$ are the lists defined in (2).

- the home-away balance decomposition constraint establishes the relationship between the actual home-away imbalance and the balance variables:

$$\forall t \in \{1 \dots n\}, \quad \text{home}_t - \text{away}_t = b_t^+ - b_t^-; \quad (34)$$

where home_t and away_t are computed as described in (5).

4.3.1 Implied constraints

Each team has exactly one defective period (proved in (1)):

$$\forall t \in \{1 \dots n\}, \quad \sum_{p=1}^P (\text{defective}_{t,p}) = 1. \quad (35)$$

4.3.2 Symmetry breaking constraints

Fix the first match maintaining the one provided by the feasible solution:

$$\text{match_period}_{1,1} = \text{True}, \quad (36)$$

$$\text{swap}_1 = \text{False}. \quad (37)$$

4.4 Validation

Experimental design

The MIP model was evaluated under two configurations: a baseline formulation without symmetry breaking constraints and an enhanced formulation incorporating first match assignment constraints (Section (4.3.2)). Each formulation was tested using two different solvers: Gurobi [4], and HiGHS [5]. All experiments were conducted on a MacBook Air equipped with an Apple M4 chip and 16 GB RAM.

Experimental results

In the following table, we report all the results obtained for the MIP model. Contrary to expectations, symmetry breaking constraints did not always improve performance and in some cases hindered it. G stands for gurobi, H stands for HiGHS.

Table 3: **Running times** in seconds (rounded down to integers) and **objective function values** for different strategies of MIP models. For every value of n and strategy the table reports: time to reach feasibility | objective value, time to reach optimality. Bold objective values denote optimal solutions.

n	G	H	G + SB	H + SB
4	UNSAT	UNSAT	UNSAT	UNSAT
6	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
8	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
10	0 1 , 0	0 1 , 0	0 1 , 0	0 1 , 0
12	0 1 , 0	2 1 , 2	0 1 , 0	1 1 , 1
14	5 1 , 5	6 1 , 6	16 1 , 17	3 1 , 3
16	36 1 , 36	58 1 , 59	49 1 , 51	11 1 , 11
18	19 1 , 20	101 1 , 101	N/A	54 1 , 54
20	N/A	N/A	N/A	269 1 , 270

5 Conclusion

We can conclude stating that the circle method was a game changer for our project.

The factorial blow-up of the problem’s search space made the experiments substantially more difficult for each increasing n , arriving up to the number of participants had in typical round-robin competitions ($n = 20$).

We also tried to study the behaviour of our models with or without symmetry breaking and implied constraints and, in contrast to the theory, we observed mixed results, depending on the different solvers that have been used.

Authenticity and Author Contribution Statement

We declare that the described work is our own and we did not copy it from someone else. We utilized GPT-5 and Gemini Pro to generate Python scripts that automated the execution of our test cases and saved the results in Excel format. Claude was adopted in the generation of some code comments.

All the authors contributed in the main idea of our approach, however we divided the job between the different approaches as follows:

- Alessandro Picardi mainly worked on the CP model,
- Davide Ligabue mainly worked on the SAT model,
- Andreea Dornescu mainly worked on the MIP model.

References

- [1] ERIK LAMBRECHTS, ANNETTE M. C. FICKER, DRIES R. GOOSSENS, FRITS C. R. SPIEKSMAN, *Round-robin tournaments generated by the Circle Method have maximum carry-over*, Springer-Verlag Berlin Heidelberg and Mathematical Optimization Society, (2017).
- [2] NIKLAS EËN, NIKLAS SÖRENSON, *Translating Pseudo-Boolean Constraints into SAT*, Journal on Satisfiability, Boolean Modeling and Computation 2, (2006).
- [3] IAIN DUNNING, STUART MITCHELL, MICHAEL O’SULLIVAN, *PuLP: A Linear Programming Toolkit for Python*, <https://optimization-online.org/2011/09/3178/>, (2011).
- [4] Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual*, <https://www.gurobi.com>, (2024).
- [5] JULIAN HALL AND CONTRIBUTORS, *HiGHS - Linear Optimization Software*, <https://highs.dev/>, (2024).